C++ FÖRDJUPNING

FÖRELÄSNING 1 - TGE

C++ FÖRDJUPNING

TGE - THE GAME ENGINE

TGA

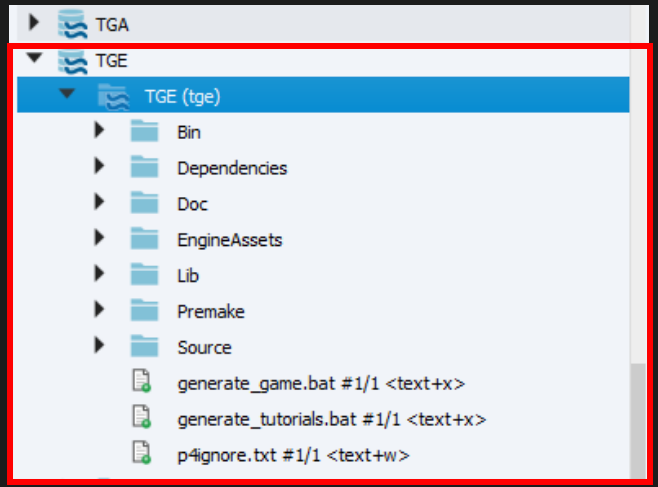
TGE

- TGE
  - Är vårt egna framework/spelmotor för att rita ut grafik och göra både 2D- och 3D-spel.
  - Det innebär att ni har tillgång till all källkod och därmed även kan göra lokala ändringar för er grupp i spelprojekten som börjar nästa termin om ni så vill.
  - Vi kan även fixa nya features och fixa buggar som uppstår.
    - I grund och botten bygger det på DirectX. Ni kan även skriva egna shaders till spelen.

TGE

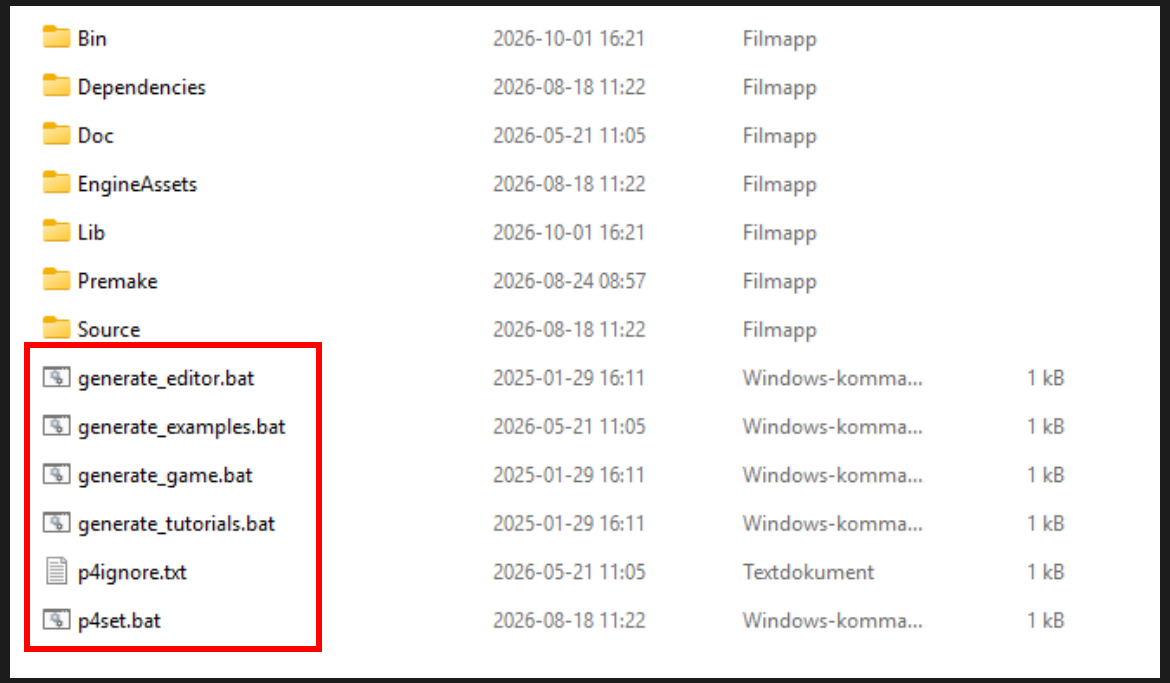
- Alla grafiska labbar och spelprojekt i årskurs ett kommer köras i TGE.
  - Ni hämtar alltid hem senaste versionen på P4V: //TGE/TGE/

- Förutom all källkod finns här även dokumentation över motorn samt exempelprojekt (kallas tutorials) för olika features.



TGE

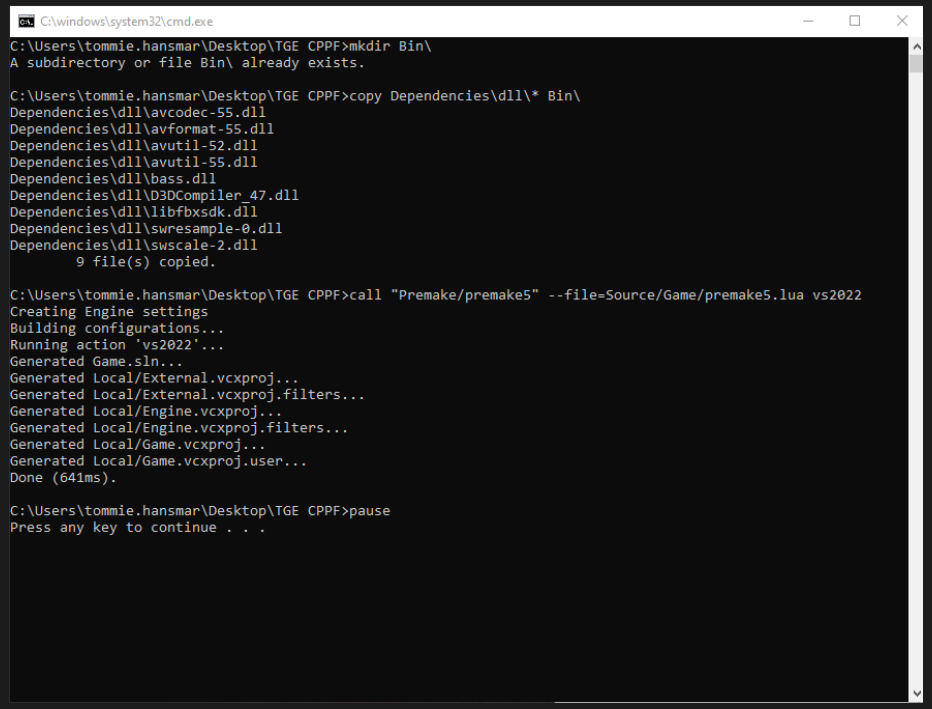
- Efter ni har dragit ner projektet skall ni först köra .bat-filerna:
  - generate_game.bat (Här gör ni spelen)
  - generate_tutorials.bat (Tutorials/Guider)
  - generate_editor.bat(själva editorn)
    - Mer om det inför P3
  - generate_examples.bat
    - Färdiga guides med exempel-kod
  - p4set.bat
    - Sätter upp en p4ignore för TGE



- Det är skript som skapar en visual studio solution och projekt

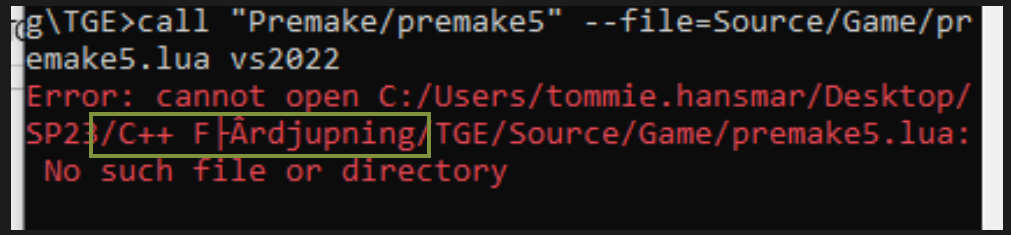
TGE

- Kör generate_game.bat
  - En Console-fönster kommer att öppnas:



  - Filer skapas, bl.a. Game.sln

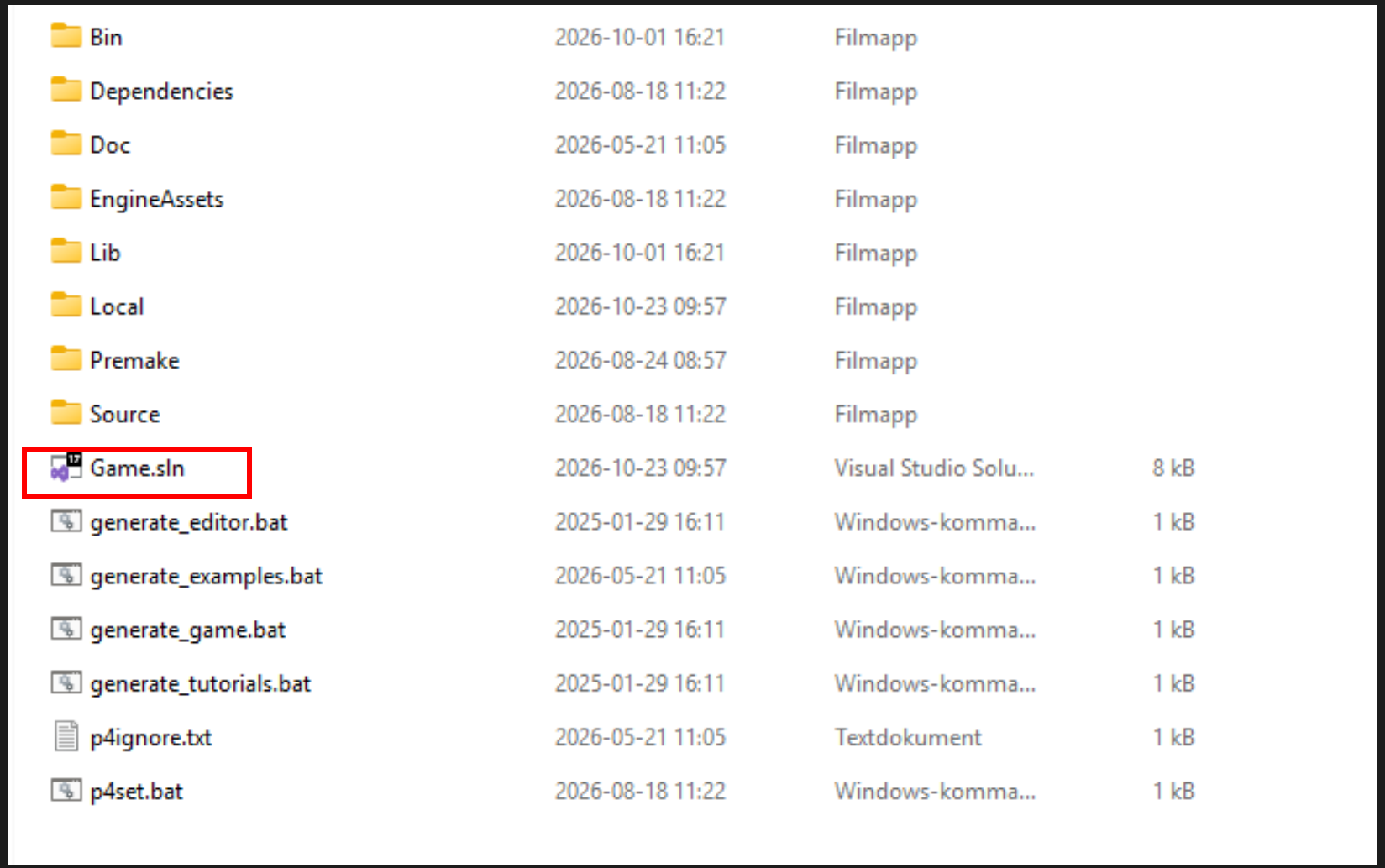
NOTERA!!!
Får ni ett Error, har ni troligtvis ert TGE-projekt i en mapp med å, ä eller ö



Undvik alltid å, ä och ö!!!

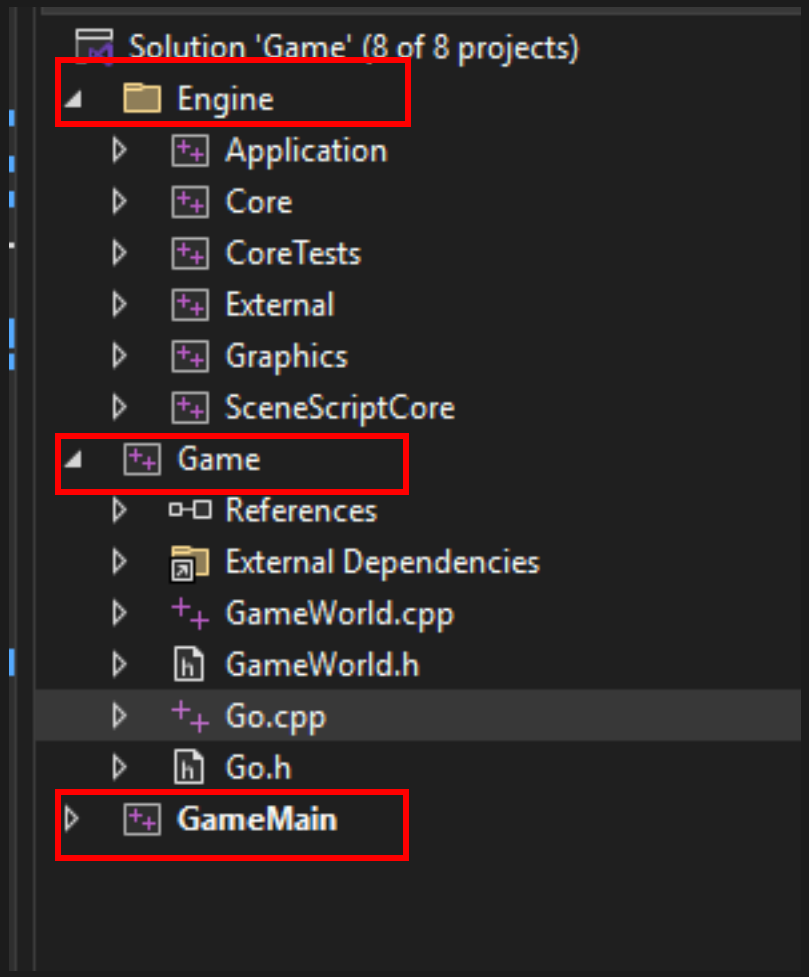
TGE

- Nu kan ni starta Game.sln



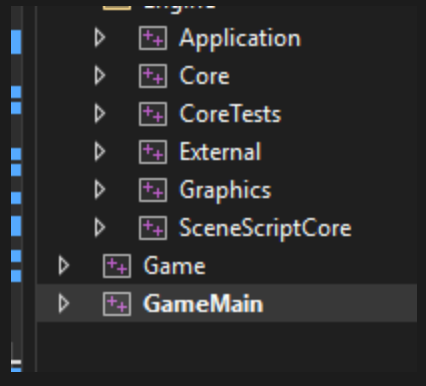
TGE

- I Solution kommer ni få fram ett par projekt
  - Engine
    - Detta är själva motorn
      - Application
        - Fönster, timer, filewatcher, etc..
      - Core
        - Själva motor-kod
      - CoreTests
      - External
        - Tredjepart bibliotek
      - Graphics
        - Grafik-relaterat som inte berör själva editorn
      - SceneScriptCore
        - Visual scripting logik
  - GameMain
    - Koppling mellan projekt
  - Game
    - Här gör ni era spel !!



TGE

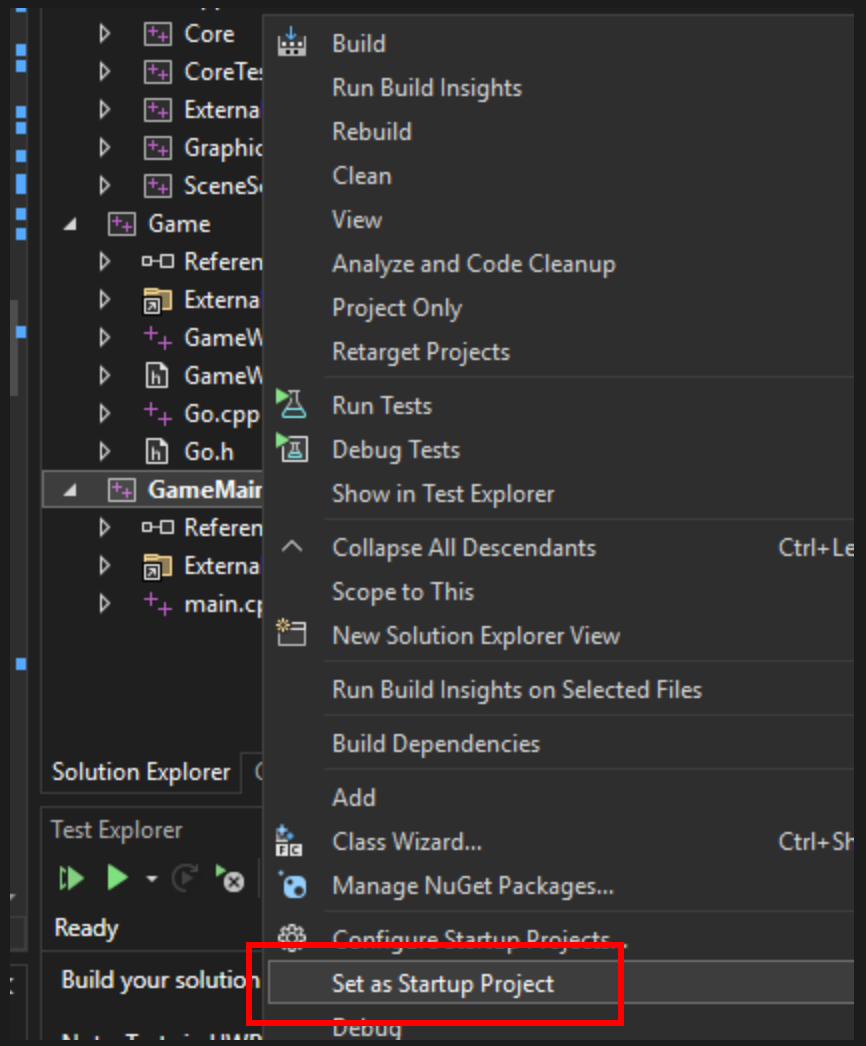
- Det är viktigt att ni sätter ”GameMain” som erat startprojekt
  - Detta skall redan vara inställt, men om ni behöver ändra:
    - Högerklicka på GameMain -> ”Set as Startup Project”
  - *Fetstil* visar vilket som är start projektet.




```cpp
#pragma once

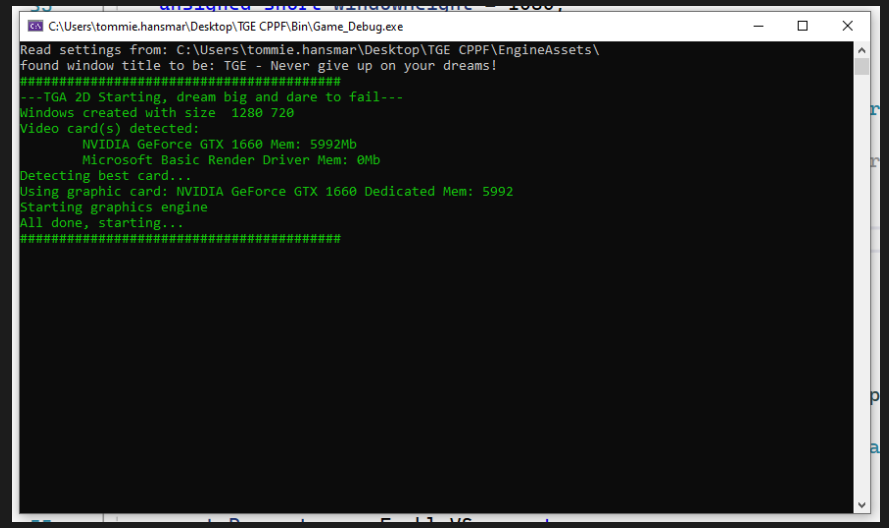
#include <cstdio>
#include "Go.h"

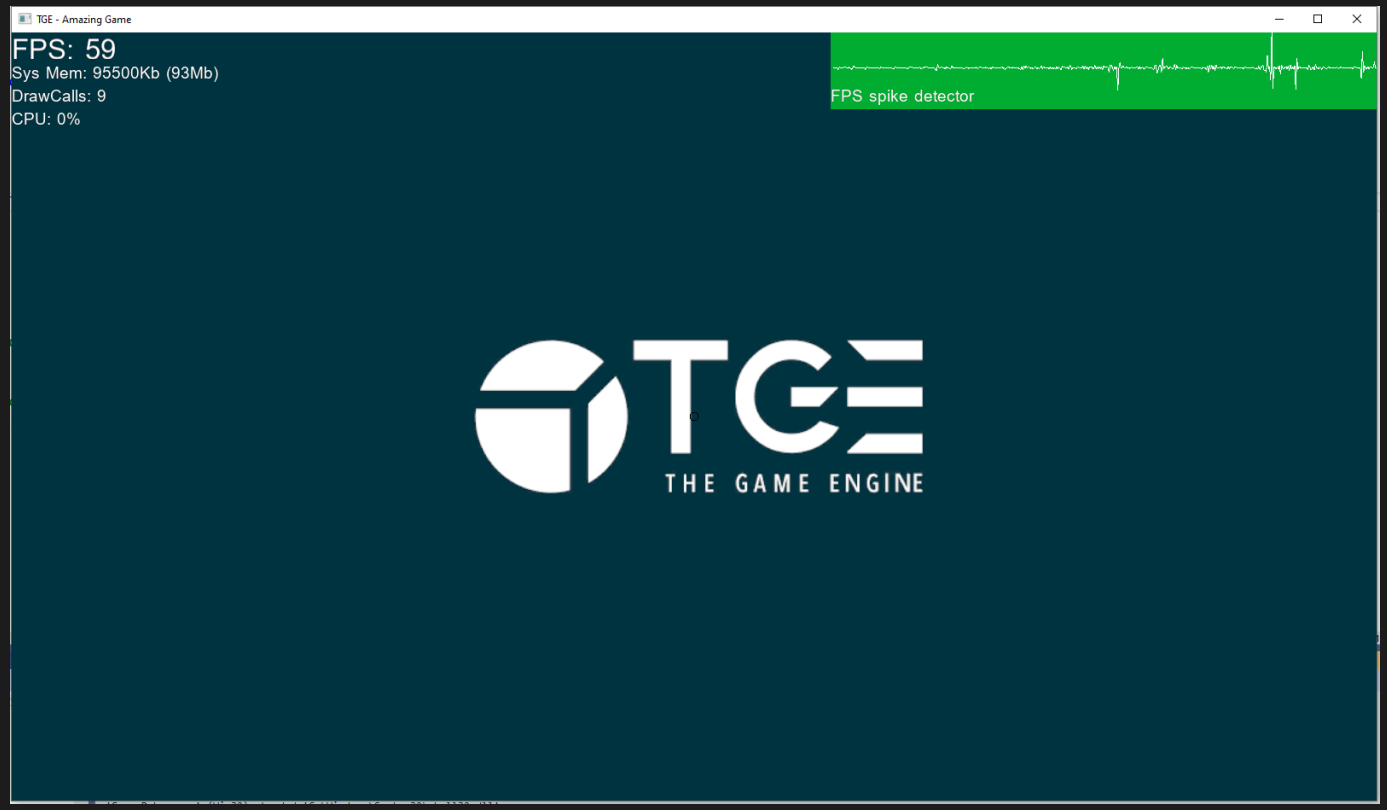
int main(const int argc, const char* argv[])
{
    if (argc > 1)
        printf("argv[1] => %s", argv[1]);
    else
        printf("argc => %d", argc);
    Go();
    return 0;
}
```



TGE

- När ni kör kommer ni få fram två fönster
  - Konsol fönster för Debug
  - Grafiskt fönster





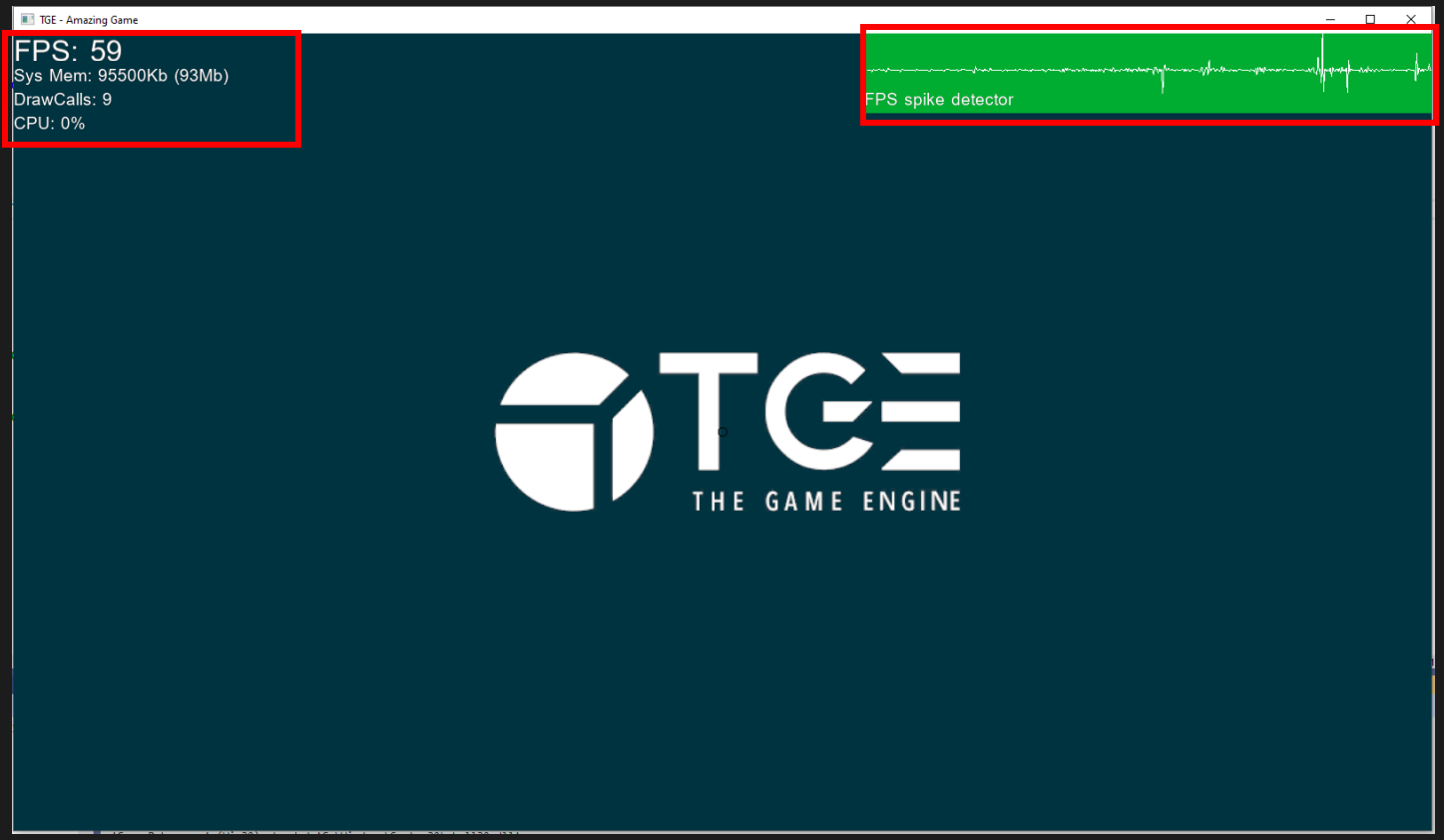
TGE

- Debug-fönstret
  - Er bästa vän
  - Felmeddelanden syns här
  - Ni kan även själv skriva ut debug-data här

```
########################################
---TGA 2D Starting, dream big and dare to fail---
Windows created with size  1280 720
Starting engine
Video card(s) detected:
        NVIDIA GeForce GTX 1660 Mem: 5992Mb
        Microsoft Basic Render Driver Mem: 0Mb
Detecting best card...
Using graphic card: NVIDIA GeForce GTX 1660 Dedicated Mem: 5992
All done, starting...
########################################
Error in texture_manager.cpp at line:143 Image file not found! sprites/error.dds
Error in texture_manager.cpp at line:189 Failed to load resource:  sprites/error.dds
DDS needs to be power of two! sprites/paddle.dds 32*160
```

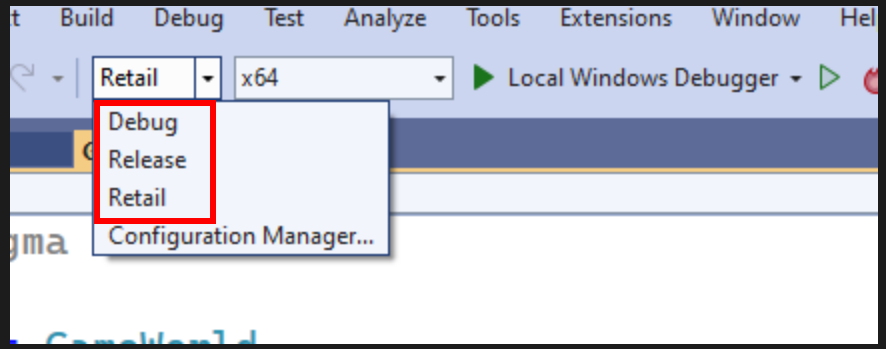
TGE

- Grafik-fönster
  - Det är här erat spel kommer synas.
  - Med lite debug-data:
    - FPS
    - Memory (RAM)
    - Draw Calls
    - CPU
    - FPS Spike Detector



TGE

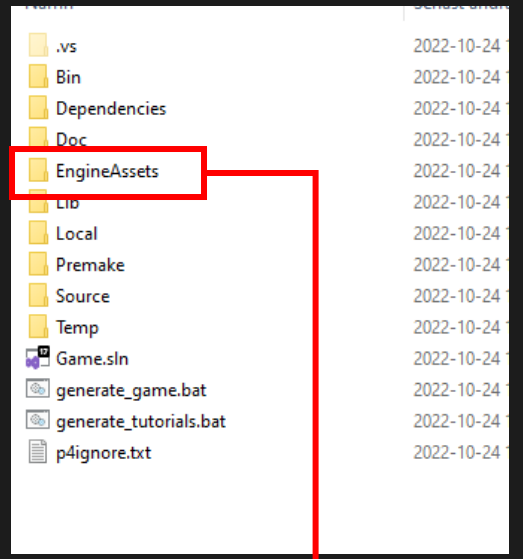
- Bygg-möjligheter
  - Debug
    - Här får vi ut all debug funktionalitet
  - Release
    - Här kan vi se prestandan med optimering
      - Debug-utskift kommer vara kvar
  - Retail
    - Optimerat spel helt utan debug
    - Det färdiga spelet!!
      - Använd denna inställning om ni vill ta bort debug-utskriften

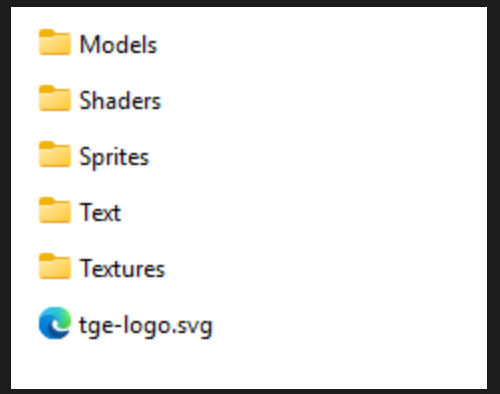


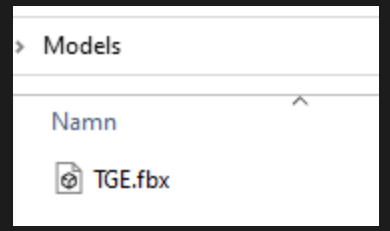


TGE

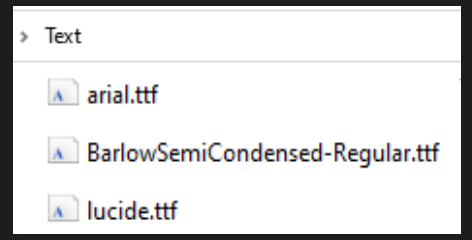
- Assets
  - TGE kan ladda lite olika filer
  - Exempel finns i EngineAssets
  - Models
    - 3D modeller
  - Sprites
    - 2D grafik
  - Shaders
    - hlsl filer för vertex och pixel shaders
  - Text
    - Fonter
  - Textures
    - Texturer
- Lägg filer för era spel i
  - Source/Game/data

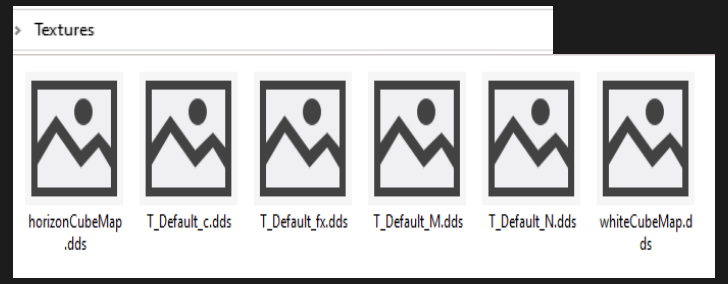












C++ FÖRDJUPNING

PREMAKE

PREMAKE

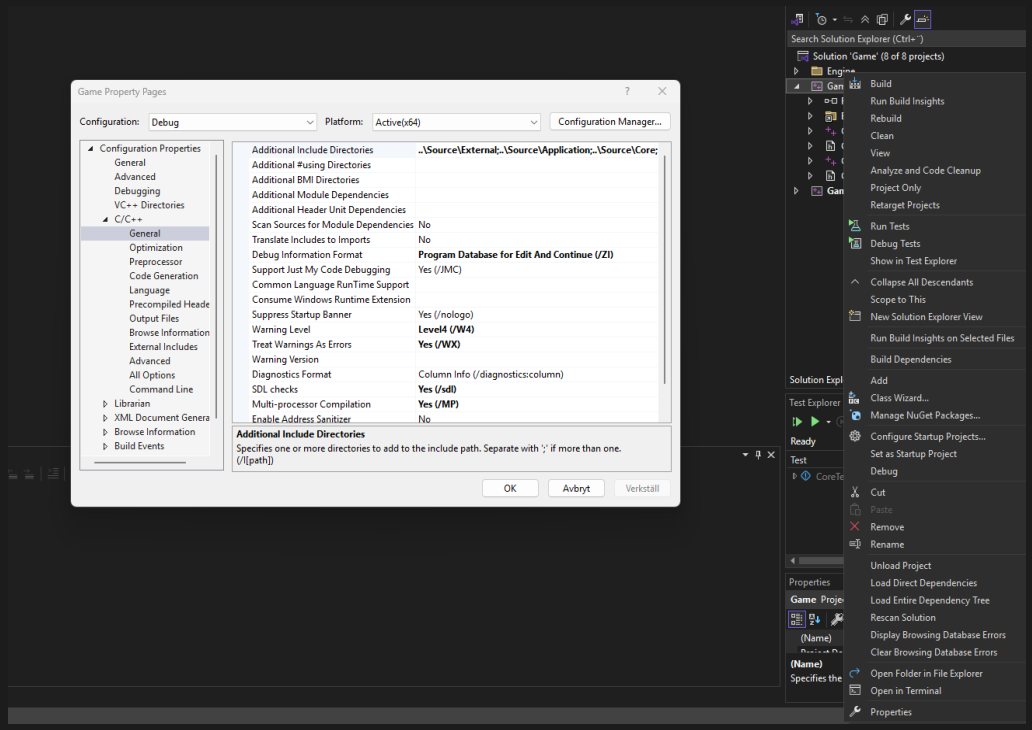


- Premake är ett verktyg för ”build configuration”
  - Kan generera Visual Studio projekt, men också projekt för andra platformar
  - Låter oss skriva regler för hur våra projekt ska vara uppsatta
  - Mer flexibelt än att använda Visual Studio projekt direkt

- Premake eller liknande system används på nästan alla studios
  - Ett annan vanlig verktyg för detta är Cmake (egentligen mest populärt)
    - Mer komplicerat
    - Svårare att använda för det vi vill få ut av det
    - Egentligen mer av en syntax, snarare än eget språk
  - Bra att se och vänja sig vid

- När vi genererar projekt med premake skriver den över projektfilerna

- OBS!
- Ändra inte projektinställningar i Visual Studio
- Vi ändrar i premake, sen genererar vi om!



PREMAKE

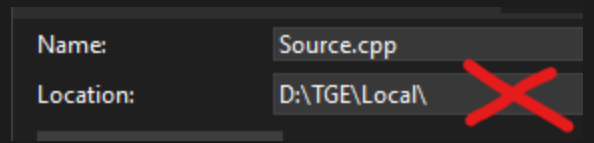
- Den använder sig av .lua script
  - Game/premake5.lua
  - Engine /premake5.lua
  - CommonUtilities/premake5.lua
  - Etc.

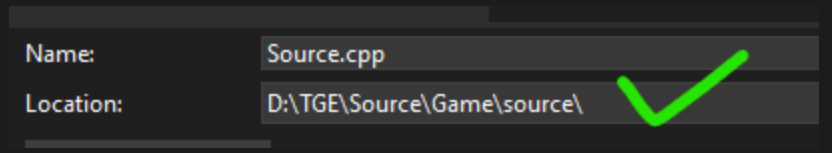
- Dessa specificerar hur projekt filerna samt solution filen konfigureras

- Så premaken sätter projekt inställningarna samt vilka projekt som skapas och läggs till i solutionen
  - Detta inkluderar de filer som inkluderas i projektet

PREMAKE

- Lägg till ny fil, två alternativ:
- Alternativ 1:
  - Lägg till från projektet Visual Studio
    - Högerklicka projekt, välj Add -> New Item
    - OBS! Ändra ”Location” för att inte lägga filerna i fel mapp
    - Borde ligga i Source/"projektet du vill lägga till filen I" (Game)/source

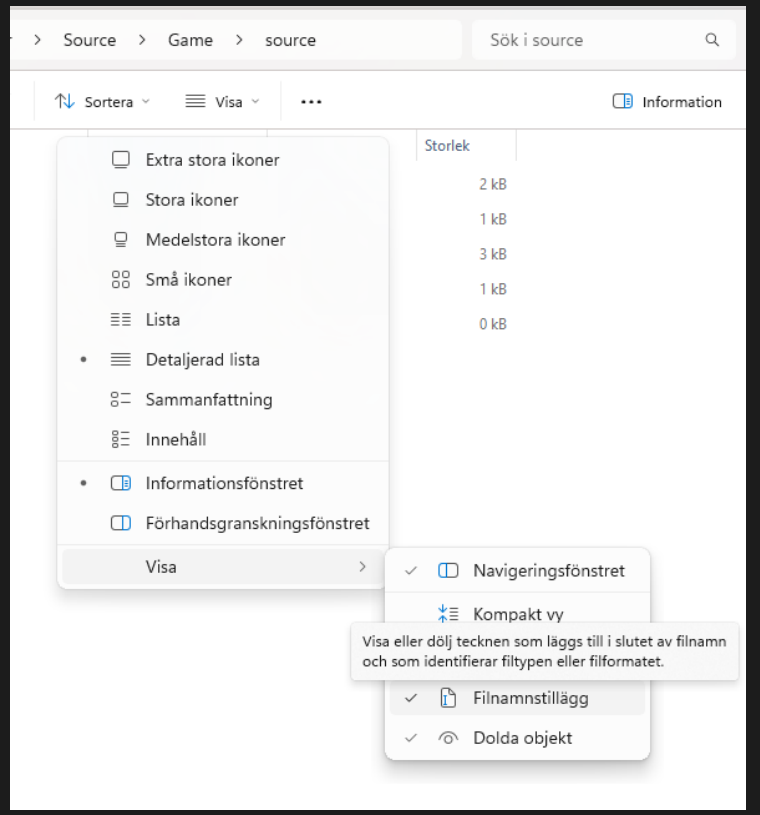


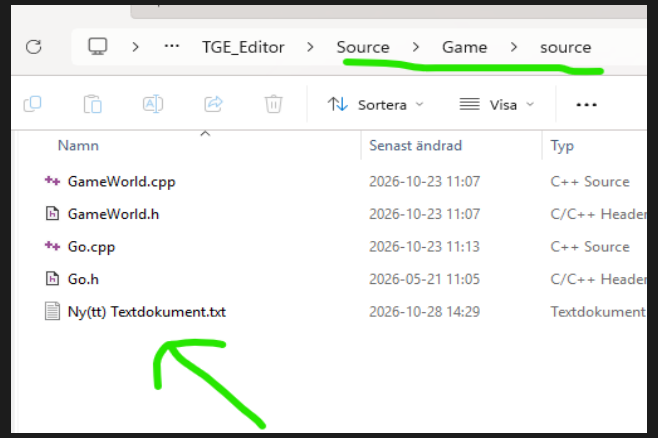


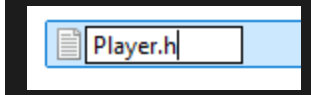
PREMAKE

- Lägg till ny fil, två alternativ:

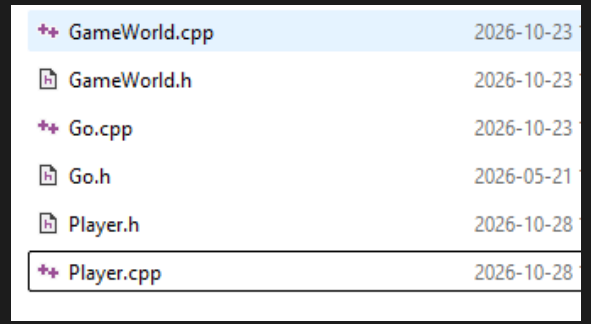
- Alternativ 2:
  - I Windows File Explorer:
    - Se till att ni har "Visa > Filnamnstillägg" checkad
    - Högerklicka i foldern, välj ”Nytt” -> ”Text dokument” och byt filextension
    - Kör om .bat-filen för att uppdatera projektet
    - Glöm inte att filerna ni skapar är helt toma
      - Headern kommer vilja ha
        - #pragma once
      - Source kommer vilja ha
        - #include "headername.h"
          - Ex: (#include "Player.h")

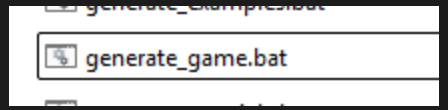












MODIFIKATIONER

- Sätta namn på spelet
  - Hitta retail uppsättningen och lägg till en target name post och sätt värdet till ert spelnamn
    - För denna posten så skriver något senare över det tidigare, så denna kommer ersätta det generella namnet för retail builden
  - `filter "configurations:Retail”`
    - `targetname("Game Name")`

- Lägga till defines
  - Vill ni ha preprocessor defines för projektet kan de läggas till här
  - `filter "configurations:Retail"`
    - `defines {"_RETAIL", "NDEBUG"}`

LÄGGA TILL FLER PROJEKT

- Ni kan lägga till fler projekt
- Exempelvis CommonUtilities
- Finns flera alternativ
  1. Ha filerna direkt i game
  2. Slänga in filerna i external och efterlikna den strukturen
  3. Inkludera lib och header filerna i premake
  4. Skapa upp ett CommonUtilities projekt med premake

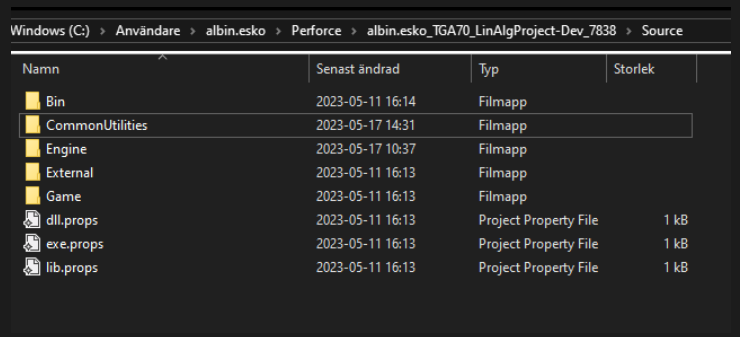
INKLUDERA BIBLIOTEK MED PREMAKE

- För att inkludera ett förkompilerat bibliotek gör vi ungefär samma som i visual studio, fast i premake:
- De är:
  1. Header directory
     - `includedirs { dirs.external, …}`
     - Kan lägga till `"%{dirs.dependencies}Library1/include"` för att kunna hämta filer från katalogen
  2. Library directory
     - `libdirs { dirs.lib, dirs.dependencies }`
     - Tips att ni bara lägger in era externa lib filer i depencencies katalogen för då är det redan uppsatt
     - Annars kan ni modifiera denna likande som för header directory
  3. Library name
     - `links {"LibFile1", "LibFile2"}`
     - Skrivs utan filändelsen (.lib)
     - Används också för att sätta referenser mellan projekt

SKAPA NYA PROJEKT MED PREMAKE

- Skapa mapp i Source för projektet
  - Lägg till en premake5.lua
  - Lägg till kod
  - Exempel finns på Omniway i kurstillbehör!

- Modifiera följande lua skript så det sätts upp rätt:
  - Premake/Common.lua
  - Source/Game/premake5.lua
  - Source/CommonUtilities/premake5.lua



PREMAKE5.LUA

```
project "CommonUtilities"
    location (dirs.projectfiles)

    language "C++"
    cppdialect "C++17"

    targetdir (dirs.lib)
    targetname("%{prj.name}_%{cfg.buildcfg}")
    objdir ("%{dirs.temp}/%{prj.name}/%{cfg.buildcfg}")

    files {
        "CommonUtilities/**.h",
        "CommonUtilities/**.hpp",
        "CommonUtilities/**.cpp",
    }

    libdirs { dirs.lib, dirs.dependencies }

    verify_or_create_settings("Game")

    ...
```

TGA

PREMAKE5.LUA

```
…

filter "configurations:Debug"
    defines {"_DEBUG"}
    runtime "Debug"
    symbols "on"
filter "configurations:Release"
    defines "_RELEASE"
    runtime "Release"
    optimize "on"
filter "configurations:Retail"
    defines "_RETAIL"
    runtime "Release"
    optimize "on"

…
```

PREMAKE5.LUA

```
…

filter "system:windows"
    kind "StaticLib"
    staticruntime "off"
    symbols "On"
    systemversion "latest"
    warnings "Extra"
    sdlchecks "true"

    flags {
        "FatalCompileWarnings",
        "MultiProcessorCompile"
    }

    defines {
        "WIN32",
        "_CRT_SECURE_NO_WARNINGS",
        "_LIB",
        "_WIN32_WINNT=0x0601",
        "TGE_SYSTEM_WINDOWS"
    }
```

KONFIGURERA FÖR COMMONUTILITIES

Common.lua:

- Lägg till

common.lua

C: > Users > albin.esko > Perforce > albin.esko_TGA70_LinAlgProject-Dev_7838 > Premake > common.lua

```
14  dirs["engine"]          = os.realpath(dirs.root .. "Source/Engine")
15  dirs["settings"]        = os.realpath(dirs.root .. "Bin/settings/")
16  dirs["engine_assets"]   = os.realpath(dirs.root .. "EngineAssets/")
17  dirs["utilities"]       = os.realpath(dirs.root .. "Source/CommonUtilities")
18
19  dirs["shaders"] = {}
20  dirs.shaders["absolute"] = os.realpath(dirs.root .. "Bin/Shaders/")
21  dirs.shaders["relative"] = "../Bin/Shaders/"
```

Game/premake5.lua:

- Lägg till

premake5.lua

C: > Users > albin.esko > Perforce > albin.esko_TGA70_LinAlgProject-Dev_7838 > Source > Game > premake5.lua

```
14  -- include for common stuff
15  include "../../Premake/common.lua"
16
17  include (dirs.external)
18  include (dirs.engine)
19  include (dirs.utilities)
20
```

- Lägg till

premake5.lua

C: > Users > albin.esko > Perforce > albin.esko_TGA70_LinAlgProject-Dev_7838 > Source > Game > premake5.lua

```
33      targetname("%{prj.name}_%{cfg.buildcfg}")
34      objdir ("%{dirs.temp}/%{prj.name}/%{cfg.buildcfg}")
35
36      links {"External", "Engine", "CommonUtilities"}
37
38      includedirs { dirs.external, dirs.engine, dirs.utilities }
39
```

PREMAKEA VIDARE

- Det finns andra saker att göra med premake
- Den kan fixa alla projekt inställningar som ni behöver för era c++ projekt
- Dokumentationen hittar ni här https://premake.github.io/docs/

C++ FÖRDJUPNING

IDAG



IDAG

- Tutorials
  - Generera solution för tutorials
  - Starta de olika tutorial-projekten (sätt dem som startup projekt)
  - Testa att ändra några värden här och där

- Game
  - Generera solution för game
  - Testa att ändra vad som ritas på skärmen på nåt sätt
  - Skapa och inludera ett nytt premake-project för Common Utilities
    - Mall finns på Omniway
  - Använd den på nåt sätt i Game
  - Rita ut en sprite som flyttar sig på nåt sätt

- Behöver ni hjälp? Fråga!
- Tips:
  - Qrikko och Björn är superduktiga med både TGE och Premake
  - Jag är "ganska" duktig på Premake, men duktig på TGE

TGA